B 2.2.4 Queues Python

B 2.2.4 Explain the concept of a queue as a “first in, first out” (FIFO) data structure Python

This section introduces the queue data structure, which operates on the FIFO principle (like a line at a store!). We'll explore key operations (`enqueue`, `dequeue`, `front`, `isEmpty`), their efficiency, and how to choose the right queue implementation for different scenarios.

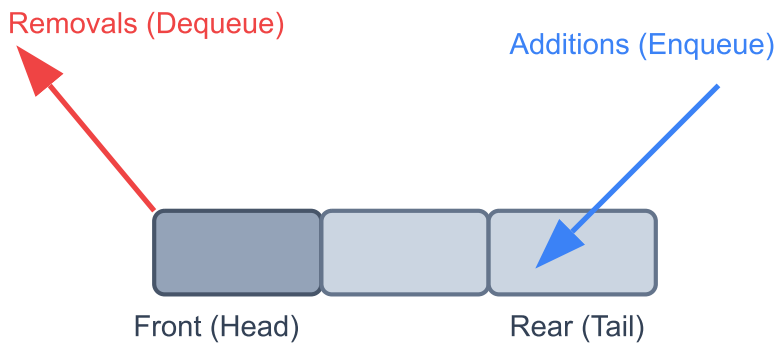
What is a Queue?

A queue is a fundamental linear data structure in computer science. Its defining principle is **FIFO**, which stands for **"First-In, First-Out."**

This means that the first item you add to the queue is always the first item you can remove.

A simple way to visualise this is to think of a checkout line at a store or a line for a roller coaster. The first person to get in line (the "front" of the queue) is the first person to be served. New people are added to the "rear" (end) of the queue.

Queue: FIFO Structure



Fundamental Queue Operations

There are four primary operations you must know for managing a queue.

1. enqueue(item)

- **What it does:** Adds a new element (an `item`) to the **rear** (end) of the queue.
- **Error Check:** If the queue is implemented with a fixed size (like an array) and is already full, attempting to `enqueue` a new item will cause a **Queue Overflow** error.

2. dequeue()

- **What it does:** Removes the element from the **front** (start) of the queue and returns that element. The queue's size decreases by one.
- **Error Check:** If you try to `dequeue` an item from an empty queue, it will cause a **Queue Underflow** error, as there is nothing to remove.

3. front() (or peek())

- **What it does:** Returns the element at the **front** of the queue *without* removing it. This is useful for checking what the next item to be dequeued is.

- **Error Check:** Just like `dequeue`, attempting to `front()` or `peek()` at an empty queue will cause a **Queue Underflow** error.

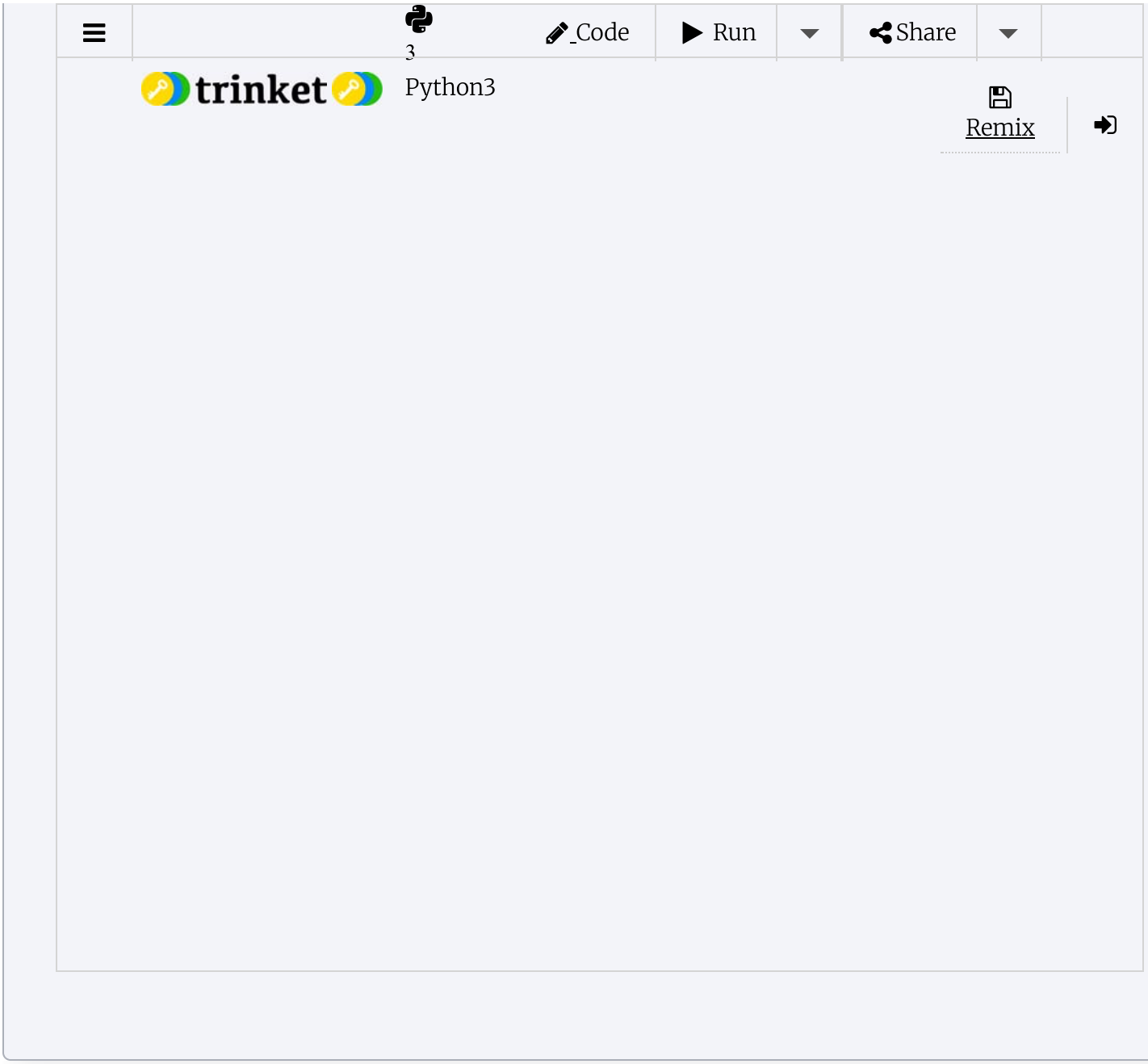
4. isEmpty()

- **What it does:** A utility operation that checks if the queue has any elements in it. It returns a boolean value (`True` if the queue is empty, `False` otherwise).
- **Why it's important:** You should always use `isEmpty()` to check the state of a queue before calling `dequeue()` or `front()` to prevent underflow errors.

Interactive Practice: List-Based Queue

Use the code editor below to practice. In Python, the best way to build a queue is with the `deque` object from the `collections` module.

- Try using `enqueue()` (which we'll code using `append()`) to add items.
- Use `dequeue()` (using `popleft()`) to remove them and see the FIFO principle.
- Use `front()` to check the front item.
- Try to `dequeue` from an empty stack to see the underflow error message.



Performance and Memory Impact

Time Complexity (Performance)

- **Good Implementation (`collections.deque`, Circular Array):** The fundamental operations are extremely fast.
  - `enqueue`, `dequeue`, `front`, and `isEmpty` all have a time complexity of `O(1)`.
  - This "constant time" efficiency is what makes queues so powerful.
- **Bad Implementation (Python `list`):**
  - `enqueue` (using `list.append()`) is fast: `O(1)`.

- `dequeue` (using `list.pop(0)`) is very slow: `O(n)`, because every other element must be shifted one position. This is why you should **never** use a `list` as a queue.

Memory Usage

The memory usage of a queue is directly proportional to the number of elements it contains.

- This is known as `O(n)` space complexity, where `n` is the number of items in the queue.
- If you use a fixed-size array, you are always using `O(CAPACITY)` memory, even if the queue is empty.

Choosing the Right Queue for a Problem

Queues are essential for any problem involving order, fairness, or processing items as they arrive.

Common Applications:

- **Print Jobs:** The first document sent to the printer is the first one printed.
- **Web Servers:** Handling incoming user requests in the order they were received.
- **Keyboard Buffer:** The first key you press is the first character to appear.
- **Breadth-First Search (BFS):** A key algorithm in graphs, used for finding the shortest path.
- **Simulations:** Modeling real-life waiting lines (like a bank or supermarket).

Which Implementation to Choose?

1. `collections.deque` **(Recommended in Python):**
   - **Pro:** Fast `O(1)` operations for both ends, flexible, resizable, and the standard Pythonic way.
   - **Con:** None for this purpose.
   - **Best for:** Nearly all general-purpose problems in Python.
2. `queue.Queue`**:**
   - **Pro:** `O(1)` operations and is "thread-safe."
   - **Con:** More overhead than `deque`.
   - **Best for:** Multi-threaded applications where different threads need to safely add/remove items at the same time.
3. **Circular Array (Fixed-Size):**
   - **Pro:** Extremely fast and memory-efficient if you know the maximum size.
   - **Con:** Fixed size. If you get it wrong, you waste memory or get overflow errors.

- **Best for:** Embedded systems or performance-critical applications where you have a known, strict limit on the number of items.

Implementing a Queue

There are a few ways to implement the queue. You will not need to know all of these for hte exam but they are good to deepen your understanding and to use as guidance for implementing in your Internal Assessment if needed.

In Python, you have two main ways to build a queue: the "recommended" way using `deque`, and the "exam-style" way using a circular array.

1. The Recommended Way (Using `collections.deque`)

For your projects, always use `deque`. Using a plain `list` is very bad for queues, because `list.pop(0)` (to remove from the front) is an **`O(n)`** operation—it has to shift every other element, making it extremely slow. `deque` is built for this and is `O(1)`.

```
from collections import deque

class Queue:
    def __init__(self):
        # We use a deque object, optimized for appends/pops
        # from both ends.
        self.queue = deque()

    # 1. enqueue(item)
    # .append() adds to the right side (the "rear")
    def enqueue(self, item):
        self.queue.append(item)
        print(f"Enqueued: {item}")

    # 2. dequeue()
    # .popleft() removes from the left side (the "front")
    def dequeue(self):
        if self.isEmpty():
            print("Queue is empty (Underflow)")
            return None

        item = self.queue.popleft()
        print(f"Dequeued: {item}")
        return item

    # 3. front()
    # We can peek at the front item by accessing index 0
    def front(self):
        if self.isEmpty():
            print("Queue is empty")
            return None

        item = self.queue[0]
        print(f"Front is: {item}")
        return item

    # 4. isEmpty()
    # A deque is "truthy" if it has items, "falsy" if empty
    def isEmpty(self):
        return not self.queue

    # 5. displayQueue()
    def displayQueue(self):
        if self.isEmpty():
            print("Queue is empty.")
            return

        # This shows the queue from front to rear
```

```
        print("Front -> ", end="")
        for item in self.queue:
            print(f"[{item}] - ", end="")
        print("<- Rear")

# --- Example Run ---
# (You can add this code after the class to test it)

# 1. Create a new queue
my_queue = Queue()
print("--- Created a new queue ---")
my_queue.displayQueue()
print("\n")

# 2. Enqueue items
print("--- Enqueuing items ---")
my_queue.enqueue("Task 1")
my_queue.enqueue("Task 2")
my_queue.enqueue("Task 3")
my_queue.displayQueue()
print("\n")

# 3. Dequeue an item
print("--- Dequeuing an item ---")
my_queue.dequeue() # Should remove "Task 1"
my_queue.displayQueue()
print("\n")

# 4. Check the front item
print("--- Checking front item ---")
my_queue.front() # Should show "Task 2"
print("\n")

# 5. Empty the queue
print("--- Emptying the queue ---")
my_queue.dequeue() # Removes "Task 2"
my_queue.dequeue() # Removes "Task 3"
my_queue.displayQueue()
print("\n")

# 6. Try to dequeue from an empty queue
print("--- Dequeuing from empty queue ---")
my_queue.dequeue() # Should print "Queue is empty (Underflow)"
```

2. The Circular Array Queue

How to implement a queue using a fixed-size array (a `list` in Python) and pointers. We must use a **Circular Queue** to get `O(1)` performance.

```
# --- Procedural "Exam-Style" Circular Queue ---

# 1. Setup the global variables
CAPACITY = 3 # A small capacity for testing
queueArray = [None] * CAPACITY
front = 0    # Index of the element to be removed
rear = -1    # Index of the last element added
size = 0     # Number of elements currently in the queue

# 2. isEmpty()
def isEmpty():
    global size
    return size == 0

# 3. isFull()
def isFull():
    global size, CAPACITY
    return size == CAPACITY

# 4. enqueue()
def enqueue(data):
    global rear, size, front, queueArray, CAPACITY
    if isFull():
        print(f"Queue Overflow: Cannot enqueue {data}")
    else:
        # 1. Calculate the new rear position (wraps around)
        rear = (rear + 1) % CAPACITY
        # 2. Insert the data
        queueArray[rear] = data
        # 3. Increment size
        size += 1
        print(f"Enqueued: {data} at index {rear}")

# 5. dequeue()
def dequeue():
    global front, size, queueArray, CAPACITY
    if isEmpty():
        print("Queue Underflow: Cannot dequeue.")
        return None
    else:
        # 1. Get the data at the current front index
        data = queueArray[front]
        queueArray[front] = None # Optional: clear the data
        # 2. Move the front pointer forward (wraps around)
        front = (front + 1) % CAPACITY
        # 3. Decrement size
        size -= 1
        print(f"Dequeued: {data} (New front index is {front})")
```

```
        return data

# 6. front()
def peek():
    global front, queueArray
    if isEmpty():
        print("Queue Empty: Cannot peek.")
        return None
    else:
        # The element to peek is always at the 'front' index
        item = queueArray[front]
        print(f"Front is: {item}")
        return item

# 7. displayQueue()
def displayQueue():
    global front, size, queueArray, CAPACITY
    if isEmpty():
        print("Queue is empty.")
        return

    print(f"Front ({front}) -> ", end="")
    # Loop 'size' times, starting from 'front'
    for i in range(size):
        # Calculate the actual index with wrap-around
        index = (front + i) % CAPACITY
        print(f"[{queueArray[index]}] - ", end="")
    print(f"<- Rear ({rear})")

# --- Example Run ---
print("\n--- Testing Circular Array Queue ---")
print(f"--- Enqueuing 3 items (Queue size is {CAPACITY}) ---")
enqueue(10)
enqueue(20)
enqueue(30)
displayQueue()

print("\n--- Attempting to enqueue to full queue ---")
enqueue(40) # Fails (Queue Overflow)

print("\n--- Dequeuing one item ---")
dequeue() # Dequeues 10
displayQueue()

print("\n--- Enqueuing one item (wraps around) ---")
enqueue(40) # Enqueues at index 0
displayQueue()

print("\n--- Emptying the queue ---")
dequeue() # Dequeues 20
```

```
dequeue() # Dequeues 30
dequeue() # Dequeues 40
displayQueue()

print("\n--- Attempting to dequeue from empty queue ---")
dequeue() # Fails (Queue Underflow)
```